Kubernetes + Helm + Dorarion

간단한 클러스터 설치 가이드

> **목표:** 공개 OCI Helm 차트로 Dorarion 을 설치하고, 외부 PostgreSQL 데이터베이스에 연결하며, 필요한 Kubernetes Secret 과 첫 번째 관리자 계정을 만든 다음 애플리케이션을 확인합니다.

- Kubernetes 클러스터에 접근하도록 설정된 터미널에서 모든 명령을 실행합니다.
- 이 가이드는 Docker Hub 의 Helm 차트 `kimc1992/dorarion` 버전 `0.0.3` 을 사용합니다.
- Dorarion 에는 Kubernetes 1.28 이상과 Helm 3 이 필요합니다.
- 이 차트는 PostgreSQL 을 설치하지 않습니다. 먼저 데이터베이스와 백업을 준비하세요.
- PDF 버전도 제공됩니다.

1. Kubernetes 및 Helm 확인

무언가를 생성하기 전에 `kubectl` 이 올바른 클러스터를 가리키는지 확인합니다.

```
kubectl config current-context
kubectl cluster-info
kubectl version
helm version
```

컨텍스트가 다른 클러스터를 가리키면 계속 진행하지 마세요. Kubernetes 서버 버전은 최소 1.28 이어야 합니다.

2. 게시된 차트 확인

Docker Hub 에서 차트 정보를 직접 확인합니다.

```
helm show chart oci://registry-1.docker.io/kimc1992/dorarion \
  --version 0.0.3
```

예상되는 차트 메타데이터는 다음과 같습니다.

- 차트: `dorarion`
- 차트 버전: `0.0.3`
- 애플리케이션 버전: `0.0.3`
- 애플리케이션 이미지: `kimc1992/dorarion-core:0.0.3`

3. PostgreSQL 준비

빈 PostgreSQL 데이터베이스와 Dorarion 전용 사용자를 만듭니다. 이 사용자는 접속할 수 있어야 하며 테이블, 인덱스, 제약 조건을 생성하고 변경하고 Dorarion 마이그레이션을 실행할 권한이 있어야 합니다.

데이터베이스 제공자가 요구하는 TLS 옵션을 연결 URL 에 적용하세요. 일반적인 URL 형식은 다음과 같습니다.

```
postgres://USER:PASSWORD@HOST:5432/DB_NAME
```

Dorarion 을 업그레이드하기 전에 항상 데이터베이스를 백업하세요. 별도의 환경에서 복원을 시험해야 합니다. Helm 차트는 PostgreSQL 을 운영하거나 백업하지 않습니다.

선택 사항: 간단한 PostgreSQL Pod 실행

PostgreSQL 이 아직 없다면 이 저장소의 단일 노드 PostgreSQL 예제를 사용할 수 있습니다. 이 예제는 10 Gi 영구 볼륨 클레임, PostgreSQL Deployment 하나, 내부 ClusterIP Service 를 생성합니다.

> **이 예제는 학습 및 소규모 테스트 설치용입니다.** 고가용성이 아니며, TLS 와 백업을 구성하지 않습니다. 프로덕션에서는 관리형 데이터베이스나 지속적으로 유지보수되는 PostgreSQL Operator 를 사용하세요.

네임스페이스와 임의의 PostgreSQL 비밀번호를 생성합니다.

```
kubectl create namespace dorarion --dry-run=client -o yaml | kubectl apply -f -

POSTGRES_PASSWORD="$(openssl rand -hex 32)"

kubectl -n dorarion create secret generic dorarion-postgresql \
  --from-literal=POSTGRES_DB=dorarion \
  --from-literal=POSTGRES_USER=dorarion \
  --from-literal=POSTGRES_PASSWORD="$POSTGRES_PASSWORD"
```

GitHub 에서 PostgreSQL 을 바로 배포하고 준비될 때까지 기다립니다.

```
kubectl apply -f \
  https://raw.githubusercontent.com/dorarion/dorarion-helm/master/examples/postgresql.yaml

kubectl -n dorarion rollout status deployment/dorarion-postgresql \
  --timeout=5m
kubectl -n dorarion get pod,service,persistentvolumeclaim \
  -l app.kubernetes.io/name=dorarion-postgresql
```

PostgreSQL 을 만든 뒤 새 터미널을 열었다면, 화면에 표시하지 않고 비밀번호를 셸 변수로 다시 가져옵니다.

```
POSTGRES_PASSWORD="$(kubectl -n dorarion get secret dorarion-postgresql \
  -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 --decode)"
```

이 예제를 사용한다면 다음 절에서 이용할 데이터베이스 URL 을 설정합니다.

```
DATABASE_URL="postgres://dorarion:${POSTGRES_PASSWORD}@dorarion-postgresql.dorarion.svc.cluster.local:5432/dorarion"
```

4. 네임스페이스 및 Secret 생성

데이터베이스 마이그레이션이 pre-install hook 으로 실행되므로 `helm install` 전에 Secret 이 있어야 합니다.

네임스페이스를 생성합니다.

```
kubectl create namespace dorarion --dry-run=client -o yaml | kubectl apply -f -
```

애플리케이션 시크릿을 생성하고 Kubernetes Secret 을 만듭니다. PostgreSQL 예제를 사용했다면 `DATABASE_URL` 이 이미 설정되어 있습니다. 외부 데이터베이스를 사용한다면 다음 값을 실제 연결 정보로 바꿔 먼저 실행하세요.

```
DATABASE_URL='postgres://USER:PASSWORD@HOST:5432/DB_NAME'
```

이제 애플리케이션 Secret 을 생성합니다.

```
: "${DATABASE_URL:?Set DATABASE_URL before continuing}"
JWT_SECRET="$(openssl rand -hex 48)"
SECRET_ENCRYPTION_KEY="$(openssl rand -base64 32)"

kubectl -n dorarion create secret generic dorarion-secrets \
  --from-literal=DATABASE_URL="$DATABASE_URL" \
  --from-literal=JWT_SECRET="$JWT_SECRET" \
  --from-literal=SECRET_ENCRYPTION_KEY="$SECRET_ENCRYPTION_KEY"

unset DATABASE_URL POSTGRES_PASSWORD JWT_SECRET SECRET_ENCRYPTION_KEY
```

> **이 값을 안전하게 보호하세요.** Git 에 커밋하거나 Helm values 에 넣거나 채팅, 스크린샷, 티켓에 게시하지 마세요. `SECRET_ENCRYPTION_KEY` 는 비밀번호 관리자나 시크릿 관리자에 별도로 백업하세요. 이 키를 잃으면 Dorarion 에 직접 저장한 연동 자격 증명을 복구할 수 없습니다.

값을 출력하지 않고 Secret 의 키 이름만 확인합니다.

```
kubectl -n dorarion describe secret dorarion-secrets
```

`DATABASE_URL`, `JWT_SECRET`, `SECRET_ENCRYPTION_KEY` 가 있어야 합니다.

5. Dorarion 설치 및 첫 번째 관리자 생성

사용자와 CI Runner 가 실제로 접근할 HTTPS 주소를 선택합니다. 첫 설치에서만 관리자 bootstrap 을 활성화하여 Dorarion 을 설치합니다.

```
helm upgrade --install dorarion \
  oci://registry-1.docker.io/kimc1992/dorarion \
```

```
  --version 0.0.3 \
  --namespace dorarion \
  --set publicUrl=https://dorarion.example.com \
  --set bootstrapAdmin.enabled=true \
  --set bootstrapAdmin.username=admin \
  --set bootstrapAdmin.email=admin@example.com \
  --wait \
  --timeout 15m
```

사용자, GitLab Runner, 보고서나 webhook 을 Dorarion 으로 보내는 시스템에서 접근할 수 있는 주소를 사용하세요. 프로덕션 설치에서는 HTTPS 를 사용해야 합니다.

설치는 다음 순서로 진행됩니다.

1. 차단 방식의 데이터베이스 migration Job 실행
2. Dorarion Deployment 와 Service 생성
3. 애플리케이션이 준비될 때까지 대기
4. 첫 번째 관리자 bootstrap Job 실행

6. 관리자 비밀번호 저장

bootstrap Job 은 생성된 비밀번호를 한 번 출력합니다. 로그를 확인합니다.

```
kubectl -n dorarion logs job/dorarion-bootstrap-admin
```

비밀번호를 즉시 비밀번호 관리자에 저장하세요. 네임스페이스를 읽을 수 있는 사용자가 로그에서 자격 증명을 가져가지 못하도록 Job 을 삭제합니다.

```
kubectl -n dorarion delete job dorarion-bootstrap-admin
```

이 데이터베이스에서는 `bootstrapAdmin` 을 다시 활성화하지 마세요. 로그인 계정이 이미 있으면 Dorarion 이 추가 bootstrap 을 거부하지만, 업그레이드에서도 명시적으로 비활성화해야 합니다.

7. 설치 확인

Helm 릴리스와 Kubernetes 워크로드를 확인합니다.

```
helm status dorarion --namespace dorarion
kubectl -n dorarion get deployment,pods,service,jobs
kubectl -n dorarion rollout status deployment/dorarion
```

마이그레이션이 실패하면 남아 있는 Job 과 로그를 확인합니다.

```
kubectl -n dorarion get jobs
kubectl -n dorarion logs job/dorarion-migrate
```

가장 흔한 원인은 잘못된 `DATABASE_URL`, 차단된 PostgreSQL 네트워크 접근, 누락된 Secret 키, 부족한 데이터베이스 권한입니다.

8. 로컬에서 Dorarion 열기

Ingress 없이 빠르게 확인하려면 ClusterIP Service 를 포트 포워딩합니다.

```
kubectl -n dorarion port-forward service/dorarion 3000:80
```

이 터미널을 열어 둔 채 다음 주소로 이동합니다.

http://127.0.0.1:3000/login

`admin` 또는 `admin@example.com` 과 6 단계에서 저장한 비밀번호로 로그인합니다. 다른 터미널에서 readiness 를 확인할 수 있습니다.

```
curl --fail http://127.0.0.1:3000/health/ready
```

9. Ingress 구성

Ingress Controller 와 DNS 가 미리 준비되어 있어야 합니다. 시크릿이 없는 로컬 values 파일 `values.local.yaml` 을 만듭니다.

```
publicUrl: https://dorarion.example.com

ingress:
  enabled: true
  className: nginx
  hosts:
    - host: dorarion.example.com
      paths:
        - path: /
          pathType: Prefix
  tls:
    - secretName: dorarion-tls
      hosts:
        - dorarion.example.com
```

관리자 bootstrap 을 활성화하지 않고 적용합니다.

```
helm upgrade dorarion \
  oci://registry-1.docker.io/kimc1992/dorarion \
  --version 0.0.3 \
  --namespace dorarion \
  --reuse-values \
  --set bootstrapAdmin.enabled=false \
  --values values.local.yaml \
  --wait \
  --timeout 15m
```

Ingress Controller 나 인증서 관리자의 방식에 따라 TLS Secret 을 구성하세요. TLS 개인 키를 values 파일에 넣지 마세요.

10. Dorarion 업그레이드

업그레이드 전에 다음을 수행합니다.

1. PostgreSQL 을 백업하고 복원이 가능한지 시험합니다.
2. 정확한 `SECRET_ENCRYPTION_KEY` 와 키 참조를 백업합니다.
3. 릴리스 노트와 마이그레이션 호환성 정책을 읽습니다.
4. 새로운 불변 차트 및 애플리케이션 버전을 선택합니다.

관리자 bootstrap 을 비활성화한 상태로 업그레이드합니다.

```
helm upgrade dorarion \
  oci://registry-1.docker.io/kimc1992/dorarion \
  --version NEW_CHART_VERSION \
  --namespace dorarion \
  --reuse-values \
  --set bootstrapAdmin.enabled=false \
  --wait \
  --timeout 15m
```

pre-upgrade migration 은 데이터베이스를 앞으로 변경합니다. Helm 릴리스를 롤백해도 데이터베이스 마이그레이션은 되돌아가지 않습니다.

11. 제거 범위

Helm 릴리스를 제거합니다.

```
helm uninstall dorarion --namespace dorarion
```

외부 PostgreSQL 데이터베이스와 수동으로 만든 `dorarion-secrets` Secret 은 삭제되지 않습니다. 재설치나 복원 가능성이 있으면 둘 다 보존하세요. 남아 있는 모든 네임스페이스 리소스를 의도적으로 제거할 때만 네임스페이스를 삭제하세요.

PostgreSQL 예제를 배포했다면 이 리소스도 Helm 릴리스와 별개입니다. 일반적인 `Delete` reclaim policy 를 사용하는 StorageClass 에서는 영구 볼륨 클레임을 삭제할 때 데이터베이스도 삭제됩니다.

```
kubectl -n dorarion delete deployment,service dorarion-postgresql
kubectl -n dorarion delete persistentvolumeclaim dorarion-postgresql
kubectl -n dorarion delete secret dorarion-postgresql
```

예제 데이터베이스를 의도적으로 폐기하고 필요한 백업을 확보한 경우가 아니면 영구 볼륨 클레임 삭제 명령을 실행하지 마세요.

빠른 요약

- Kubernetes 컨텍스트와 버전을 확인했습니다.
- Docker Hub 에 게시된 OCI 차트를 확인했습니다.
- 외부 PostgreSQL 데이터베이스와 백업을 준비했습니다.
- 또는 테스트용 단일 노드 PostgreSQL 예제를 배포했습니다.
- 네임스페이스와 필수 Kubernetes Secret 을 만들었습니다.
- 첫 설치에서만 bootstrap 을 활성화하여 Dorarion 을 설치했습니다.
- 생성된 관리자 비밀번호를 저장하고 bootstrap Job 을 삭제했습니다.
- Deployment, readiness, 브라우저 접근을 확인했습니다.
- 이후 업그레이드에서는 bootstrap 을 비활성화합니다.

공식 참고 자료

- Dorarion Helm 소스
- Docker Hub 의 Dorarion 차트
- Helm 설치
- Helm OCI Registry
- kubectl 설치